

Kev and CLM: two ways to decide

Difference	Kev (small models)	CLM
Inputs	State, question and options together	State and actions encoded separately
Trainable	LoRA + pointer head	Projection heads only
Features	Encoder behavior changes	LLM features stay fixed

Our lab follows Kev's LoRA + pointer-head recipe.

Why this lab uses Kev

Criterion	Kev	CLM
Strength	Adapts encoder features Small released models	Reuses action embeddings Cheap repeated head training
Tradeoff	Backpropagates through the encoder for LoRA	Depends on fixed features No joint token interaction
Lab fit	Main lab recipe Full base stage is public	Useful comparison Fixed-feature head training

Open training stages, LoRA adaptation and a small pointer head.

Training and serving cost

Work	Key LoRA	CLM heads
Training	Encoder forward + backward for adapters and head	Extract fixed features then train two heads
Repeat runs	Repeat encoder compute for every experiment	Reuse feature cache Train heads only
Serving	Encode state and options Reuse a shared prefix	Encode changing state Reuse action vectors

Total cost = feature extraction + training + serving.

Performance: compare like with like

Published result	Setup	What it supports
CLM: 36-44 ms	RTX 4090; cached actions 1-1,024 candidates	Low added cost as the action set grows
Kev-4B: 12 ms	RTX PRO 6000 One question; median	Fast serving in that llama.cpp setup
Kev vs CLM	No matched Pac-Man comparison established	Measure skill, latency and memory together

Different hardware and tasks: these are not a head-to-head result.

Serving: models and readouts

Model	Readout	Useful strength	Main tradeoff
Julia-1 144M	Option-marker head	Small; multilingual 50+ languages	Limited knowledge/reasoning Private training pipeline
Laya 421M	Option-marker head	Small; designed for specialization	Fully fine-tuned encoder English root: 512 tokens
Kev-4B	Pointer head	Adaptable features Open training code	LoRA changes features Options encoded each step
lev 4B	Vocabulary label logits	Existing LM readout Many label codes	LoRA; two choice orders Extra head not converted
OpenJev 27B	Vocabulary label logits	Text and images Agent decisions	Larger model Non-commercial weights

Watch the model, then read the maths

[CLM animation](#) | [interactive controls](#)

Key: joint encoding $\rightarrow$ LoRA $\rightarrow$ pointer scores

$$z_j = q^T k_j / \sqrt{256}; p_j = \exp(z_j) / \sum_k \exp(z_k)$$

Maths: [softmax](#) | [CLM head training](#) | [LoRA](#)

p_j compares legal options; it is not win odds

Our 90-minute route

0–20 Compare models, animation and play

20–30 Review Pac-Man planning labels

30–60 Fine-tune LoRA and the pointer head

60–80 Evaluate the same boards and seed

80–90 Explain and submit CP1

Prepare Colab before class

RTX PRO 6000 Blackwell: 96 GB target

Pinned PyTorch/CUDA; BF16 autocast

Optimized FLA, convolution and fused AdamW

Pac-Man: batch 4 × accumulation 2

Checkpointing on; row budget 0

Five stages, five separate adapters

- 1 Decisions: 12,576 records $\times$ 2 epochs
- 2 Dates/evidence: 1,425 + 2,000 replay
- 3 Documents: 5,219 + 2,000 replay
- 4 Skills/devtools: 11,320 + 4,000 replay
- 5 Pac-Man: 4,096 + 2,000 replay; 1 epoch

Interactive play: meet the four ghosts

Classic maze, assets and native game mechanics

Blinky pursues; Pinky targets ahead

Inky uses Blinky; Clyde retreats when close

Release counters and modes change the threat

Badge: Skills LoRA before; Pac-Man LoRA after

Create labels from native game states

4,096 train; 256 development; 256 evaluation

Predict the four ghosts with native game rules

Search 20 moves; eight paths per first action

Rank survival, score, pellets and progress

Review labels; separate episodes across splits

CP1: fine-tune and evaluate Pac-Man

Review 4,096 planning-labelled game states

Warm-start Skills; 2,000 decision replay records

One epoch: 762 updates at learning rate $2e-5$

Train rank-16 LoRA and 256-dim pointer head

Compare Skills and task adapters; save evidence

Training curves, recovery and backup

33.8M trainable parameters: LoRA + head

Base matrices stay fixed; LoRA changes features

TensorBoard: loss, gradients, step time and VRAM

Recovery saves optimizer, scheduler and RNG

Drive enabled? Back up after each saved snapshot

Evaluate decisions and gameplay

Skills vs Pac-Man: the same 256 held-out boards

Report strict and tie-aware teacher agreement

Inspect search regret and immediate captures

Same seed: compare pellets, score and revisits

Save paired predictions, latency and GPU cost

Submit CP1: show what changed

Explain a changed move or remaining mistake

Connect the move to the ghost behaviour

Show trainable parameters and training curves

Save task adapter and paired evaluation results

Submit the notebook, runtime and compute use